

Extração de Dados com Python e IA Generativa

Do dado bruto ao dataset pronto para análise

Projeto: Natural ou Fake Natty? Como Vencer na Era das IAs Generativas

Autor: Carlos Silva

Tecnologias: Python, Requests, BeautifulSoup, Pandas e APIs

Resumo

Este e-book apresenta um fluxo prático para extração, tratamento e organização de dados usando Python, com apoio de Inteligência Artificial Generativa. O objetivo é mostrar como transformar informações disponíveis em APIs, páginas web e arquivos em dados estruturados para análises, dashboards e projetos de Engenharia de Dados.

1. Introdução à Extração de Dados

O que é extração de dados?

Extração de dados é o processo de coletar informações de diferentes fontes e transformá-las em um formato utilizável. As fontes podem ser APIs, bancos de dados, páginas web, arquivos CSV, Excel, JSON ou sistemas corporativos.

Por que isso é importante?

Em projetos de dados, a qualidade da análise depende diretamente da qualidade e disponibilidade dos dados. Um pipeline de extração bem construído reduz trabalho manual, aumenta a rastreabilidade e permite automatizar processos repetitivos.

2. Principais Fontes de Dados

APIs

APIs são uma das formas mais estruturadas de obter dados. Normalmente retornam JSON e permitem integração entre sistemas.

Web

Quando permitido pelos termos de uso do site, páginas públicas podem ser processadas para extrair informações estruturadas.

Arquivos

CSV, Excel e JSON continuam sendo fontes muito comuns em rotinas corporativas e projetos de dados.

Bancos de dados

SQL Server, PostgreSQL e outros bancos permitem consultas diretamente na fonte, quando o acesso é autorizado.

3. Python para Extração

Bibliotecas essenciais

Requests é útil para HTTP e APIs; BeautifulSoup para parsing de HTML; Pandas para transformação e análise; lxml pode ser usado em cenários de parsing mais avançados.

Exemplo de API

O código abaixo demonstra uma requisição simples a uma API pública e a conversão da resposta para DataFrame.

```
import requests
import pandas as pd

url = "https://api.exemplo.com/dados"
```

```
response = requests.get(url, timeout=30)
response.raise_for_status()

dados = response.json()
df = pd.DataFrame(dados)

print(df.head())
```

4. Web Scraping na Prática

Fluxo básico

1) identificar a página; 2) verificar permissões e termos; 3) enviar a requisição; 4) interpretar HTML; 5) localizar elementos; 6) normalizar os dados; 7) salvar o resultado.

Boas práticas

Use limites de requisição, identifique erros HTTP, evite sobrecarregar servidores e prefira APIs oficiais quando existirem.

```
import requests
from bs4 import BeautifulSoup

url = "https://exemplo.com"
html = requests.get(url, timeout=30).text

soup = BeautifulSoup(html, "html.parser")
titulos = [h.get_text(strip=True) for h in soup.select("h2")]

print(titulos)
```

5. Tratamento e Qualidade

Dados brutos não são dados prontos

Após a extração, é necessário tratar tipos, duplicidades, valores ausentes, nomes de colunas e formatos.

Validações importantes

Quantidade de registros, chaves únicas, valores nulos, tipos de dados, datas e regras de negócio devem ser verificadas.

6. IA Generativa como Copiloto

Onde a IA ajuda

Modelos generativos podem ajudar a criar estruturas de código, explicar erros, sugerir transformações, documentar pipelines e gerar testes. A validação humana continua essencial.

Exemplo de prompt

Peça à IA para criar uma função Python que receba uma URL de API, trate erros HTTP, valide o JSON retornado e converta os registros em um DataFrame Pandas, explicando cada etapa.

7. Pipeline de Extração

Arquitetura simples

Fonte → Extração → Validação → Transformação → Armazenamento → Consumo.

Evolução para Engenharia de Dados

Em projetos maiores, esse fluxo pode evoluir para jobs agendados, containers, logs, orquestração, armazenamento em cloud, camadas Bronze/Silver/Gold e monitoramento.

8. Ética, Segurança e Governança

Extração responsável

Nem todo dado acessível tecnicamente deve ser coletado. Respeite legislação aplicável, termos de uso, políticas de acesso, privacidade e finalidade do projeto.

Segurança

Nunca coloque tokens, senhas ou chaves de API diretamente no código versionado. Utilize variáveis de ambiente ou um gerenciador de segredos.

9. Projeto Prático

Desafio

Construir um pequeno pipeline que consuma uma API pública, transforme os dados com Pandas e gere um CSV final.

Entregáveis

Código Python, requirements.txt, README, dataset gerado, validações e uma breve documentação do processo.

Etapa	Resultado
Extração	JSON obtido da API
Validação	HTTP, campos e registros verificados
Transformação	DataFrame limpo
Saída	CSV pronto para análise

10. Conclusão

Da coleta ao valor

Extrair dados é apenas o primeiro passo. O verdadeiro valor aparece quando dados confiáveis são transformados em informação útil para decisões, produtos e automações.

Checklist do Projeto

- Definir a fonte dos dados
- Verificar autorização e termos de uso
- Criar ambiente Python
- Implementar extração
- Tratar erros e dados inválidos
- Normalizar os dados
- Validar qualidade
- Salvar dataset
- Documentar o projeto
- Versionar no GitHub

Conclusão

A combinação de Python, técnicas de extração e IA Generativa permite acelerar a construção de soluções de dados sem abandonar boas práticas de engenharia. Use a IA como copiloto, valide o resultado e mantenha foco em qualidade, segurança, ética e reprodutibilidade.